Plant Disease Classification Using Deep Learning

A Comparative Study of a Custom Convolutional Neural Network and ResNet18 on the PlantVillage Dataset

Submitted by: Anuja Kumari, B.Tech Computer Engineering, South Asian University, New Delhi

Date: October 2026

Abstract

This project builds an image-classification system that identifies plant-health categories from leaf images. The task is framed as 38-class classification on the PlantVillage colour dataset (54,305 images before cleaning). Two models were implemented in PyTorch and trained on a GPU: a custom four-block convolutional neural network (CNN) with 399,142 parameters, and a ResNet18 initialised with ImageNet weights, in which only the new final layer was trained. Before evaluation, five exact duplicate images shared between the official training and test splits were removed from the test set, leaving 10,704 test images. On this test set, the custom CNN reached 97.73% accuracy, 0.075071 test loss and 96.85% macro F1-score; ResNet18 reached 95.91% accuracy, 0.129822 test loss and 94.90% macro F1-score. Test loss was calculated by evaluating each saved best checkpoint on the same cleaned test set using CrossEntropyLoss. The comparison applies only to the specific training configurations used here, and the results do not show performance on real-world field photographs.

1. Introduction

1.1 Problem statement

Plant diseases can cause substantial losses in crop quality and productivity. Identifying a disease from a leaf photograph is difficult because different diseases can produce similar spots, discolouration and damage. This project develops a deep-learning classifier that assigns a leaf image to one of 38 plant–condition classes from the PlantVillage dataset. Two approaches are compared:

1. A **Custom CNN** designed and trained from scratch for this task.
2. **ResNet18**, initialised with pretrained ImageNet weights and adapted to 38 classes.

1.2 Objectives

- Obtain and inspect the PlantVillage colour-image dataset.
- Analyse class distribution and image quality.
- Detect exact duplicate images shared between the official training and test splits and remove the duplicate test entries.
- Create reproducible training, validation and test metadata.
- Resize and normalise images and apply augmentation to training images.
- Implement a custom CNN and a ResNet18 transfer-learning model.

- Train both models on a GPU, saving checkpoints and training histories.
- Evaluate both models with overall and class-wise metrics, and inspect misclassified images.
- Save the comparison results and learning curves for reporting.

1.3 Motivation

Automated plant-disease classification can give farmers, agricultural workers and researchers a consistent first screening from a photograph. This project asks whether a small custom CNN can classify PlantVillage leaf images as well as a pretrained ResNet18. It is an experimental image-classification study, not a validated agricultural diagnostic product.

2. Dataset

2.1 Source

- **Dataset:** PlantVillage, colour-image collection.
- Source: Hugging Face dataset repository mohanty/PlantVillage; the dataset is associated with the PlantVillage study [1].
- **Downloaded:** `data.zip` (about 2.03 GB), extracted to `data/images`, together with the official split lists (`color_train.txt`, `color_test.txt`) and `leaf-map.json`.

2.2 Size and splits

The original collection holds 54,305 images in 38 classes. Five exact duplicates were found across the official training and test lists, and the five test entries were excluded. The final dataset has 54,300 images.

Split	Images	Share of final dataset
Training	39,236	72.26%
Validation	4,360	8.03%
Test	10,704	19.71%
Total	**54,300**	**100%**

The splitting procedure was:

1. Read the official training and test image lists.
2. Compute MD5 hashes to find exact duplicates across the two lists.
3. Remove the five affected test entries.
4. Keep the remaining official test images as the final test set.
5. Split the official training images (43,596) into training and validation sets with `train_test_split(test_size=0.10, random_state=42, stratify=label)`.

The test set is the official PlantVillage test split after duplicate removal, not a random split. All 38 classes are present in all three splits.

2.3 Image properties

- Original size: 256 × 256 pixels for all 54,305 images.
- Colour modes: 54,304 RGB images and one RGBA image (converted to RGB on loading).
- Unreadable images: none.
- Model input: 224 × 224 × 3 tensors of type `float32`.

2.4 Classes

The 38 classes keep the dataset's original label strings.

Crop	Classes
Apple	Apple scab; Black rot; Cedar apple rust; healthy
Blueberry	healthy
Cherry (including sour)	Powdery mildew; healthy
Corn (maize)	Cercospora leaf spot / Gray leaf spot; Common rust; Northern Leaf Blight; healthy
Grape	Black rot; Esca (Black Measles); Leaf blight (Isariopsis Leaf Spot); healthy
Orange	Haunglongbing (Citrus greening)
Peach	Bacterial spot; healthy
Pepper, bell	Bacterial spot; healthy
Potato	Early blight; Late blight; healthy
Raspberry	healthy
Soybean	healthy
Squash	Powdery mildew
Strawberry	Leaf scorch; healthy
Tomato	Bacterial spot; Early blight; Late blight; Leaf Mold; Septoria leaf spot; Spider mites (Two-spotted spider mite); Target Spot; Tomato Yellow Leaf Curl Virus; Tomato mosaic virus; healthy

2.5 Class imbalance

The dataset is strongly imbalanced. In the original collection the largest classes are Orange Huanglongbing (5,507 images), Tomato Yellow Leaf Curl Virus (5,357) and Soybean healthy (5,090). The smallest are Potato healthy (152), Apple Cedar apple rust (275) and Peach healthy (360). Because of this imbalance, macro-averaged precision, recall and F1-score are reported alongside accuracy, since they give every class equal weight.

3. Exploratory Data Analysis

3.1 Analyses performed

Analysis	Result
Total images	54,305
Number of classes	38
Image dimensions	256 × 256 for all images
Colour modes	54,304 RGB, 1 RGBA
Unreadable images	None
Exact duplicates within the full dataset	21 duplicate groups
Exact duplicates shared by training and test	5 (3 Tomato Late blight, 2 Tomato healthy)
Missing image paths	None
Class coverage in final splits	All 38 classes in all splits

Sample images were inspected visually, and augmentation outputs were visualised during preprocessing. Leaf colour, spots and damage differ visibly between classes, while backgrounds, lighting and shadows vary between images. These observations motivated the use of CNNs and data augmentation.

3.2 Duplicate handling

The five training/test duplicate pairs had matching labels. The five test entries were removed, giving the cleaned test set of 10,704 images. The EDA also found 21 exact duplicate groups across the complete collection. The recorded outputs do not show that every within-split duplicate was removed; the confirmed correction is the removal of the five test entries that duplicated training images.

3.3 Leaf-map metadata

`leaf-map.json` (40,328 entries) was downloaded and checked against the official path lists. No keys matched those paths in that check, and the leaf map was not used for training or for removing images.

4. Preprocessing

4.1 Resizing and colour conversion

Every image is converted to RGB and resized to 224 × 224, giving tensors of shape (3, 224, 224).

4.2 Normalisation

ImageNet statistics were applied after `ToTensor()`: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).

4.3 Data augmentation

The two training notebooks used different augmentation pipelines.

Transformation	Custom CNN	ResNet18
Resize	224 × 224	224 × 224
Random horizontal flip	p = 0.5	p = 0.5
Random rotation	15°	10°
Colour jitter	brightness, contrast and saturation 0.15	not used
ToTensor + Normalize	yes	yes

Validation and test images use only resizing, tensor conversion and normalisation, with no random augmentation.

4.4 Dataset class and label mapping

A custom `PlantVillageDataset` class reads images from the metadata DataFrame, converts them to RGB, applies the transform, and returns the image with an integer label. Class names were sorted once from the training set to build a single `class_to_idx` mapping, which was reused for all splits.

4.5 DataLoaders

Setting	Training	Validation	Test
Batch size	16	16	16
Shuffle	Yes	No	No
Workers	0	0	0
Images	39,236	4,360	10,704
Batches	2,453	273	669

Sample batches were checked for shape, data type, finite values and valid label indices, and all checks passed.

5. Custom CNN Architecture

5.1 Overview

The `PlantDiseaseCNN` model has four convolutional blocks that raise the channel count from 3 to 256, followed by adaptive average pooling and a small classifier. Each block is Convolution (3 × 3, padding 1), Batch Normalisation, ReLU and Max Pooling (2 × 2).

5.2 Layer-by-layer description

Stage	Operations	Channels	Output size
Input	RGB image	3	224 × 224
Block 1	Conv, BatchNorm, ReLU, MaxPool	32	112 × 112
Block 2	Conv, BatchNorm, ReLU, MaxPool	64	56 × 56
Block 3	Conv, BatchNorm, ReLU, MaxPool	128	28 × 28
Block 4	Conv, BatchNorm, ReLU, MaxPool	256	14 × 14
Pooling	Adaptive average pool to (1, 1)	256	1 × 1
Classifier	Flatten, Dropout (p = 0.4), Linear(256, 38)	38	38 logits

The network has **399,142 parameters**, all trainable.

5.3 Role of each component

- **Convolutional layers** learn local patterns such as edges, textures and spots.
- **Batch normalisation** stabilises intermediate activations.
- **ReLU** adds non-linearity.
- **Max pooling** halves spatial resolution while keeping strong responses.
- **Increasing channel counts** let deeper blocks represent more varied features.
- **Adaptive average pooling** reduces each feature map to one value, giving a fixed 256-feature vector.
- **Dropout (0.4)** reduces overfitting in the classifier.
- **Final linear layer** outputs one score for each of the 38 classes.

The notebooks confirm this configuration but do not record experiments that justify each channel count or the dropout value, so these choices should not be described as separately optimised.

6. ResNet18 Transfer Learning

6.1 Pretrained model and modification

ResNet18, introduced by He et al. [2], was loaded from torchvision.models with ResNet18_Weights.DEFAULT (ImageNet weights, about 44.7 MB download). Its 1,000-class output layer was replaced with Linear(512, 38).

6.2 Frozen and trainable layers

All pretrained parameters were frozen, and only the new final layer was trained. The frozen backbone's convolution and batch-normalisation modules were kept in evaluation mode so their running statistics did not change.

Parameter category	Count
Total parameters	11,196,006
Trainable parameters	19,494

6.3 Fine-tuning stages

The workflow was: load pretrained weights, replace the final layer, freeze the backbone, train the new layer for up to 10 epochs, monitor validation loss and accuracy, and save the best checkpoint by validation loss. A later stage that unfroze the backbone was **not done**, so this is feature extraction with a trained classifier head rather than full fine-tuning.

7. Training Setup

7.1 Environment and reproducibility

Both models were trained on an NVIDIA GeForce RTX 3050 4 GB Laptop GPU (CUDA) under Windows, with Python 3.12.15, PyTorch 2.13.0+cu130 and Torchvision 0.28.0+cu130. The random seed was 42 (Python, NumPy and PyTorch, including CUDA). Seeding improves reproducibility but does not guarantee fully deterministic GPU operations.

7.2 Hyperparameters

Setting	Custom CNN	ResNet18
Loss	CrossEntropyLoss	CrossEntropyLoss
Optimiser	Adam	AdamW (final layer only)
Initial learning rate	0.001	0.001
Weight decay	0.0001	0.0001

Setting	Custom CNN	ResNet18
Scheduler	ReduceLROnPlateau (min, factor 0.5, patience 2)	ReduceLROnPlateau (min, factor 0.5, patience 1)
Maximum epochs	15	10
Early-stopping patience	4 epochs	3 epochs
Batch size	16	16
Device	CUDA	CUDA

Both loops monitored validation loss and saved the best checkpoint. Neither run triggered early stopping: the custom CNN ran to its 15-epoch limit and ResNet18 ran to its 10-epoch limit.

7.3 Custom CNN training history

The saved history contains epochs 2 to 15. The notebook restored epoch-1 metrics from an earlier run and resumed from epoch 2, so no newly logged epoch-1 row exists. The learning rate stayed at 0.001 throughout. The best checkpoint, chosen by lowest validation loss, was saved at **epoch 13**.

Epoch	Train loss	Train acc.	Val loss	Val acc.	Time (min)
2	0.7685	76.28%	0.3626	89.27%	13.83
3	0.5682	82.16%	0.2937	90.92%	12.45
4	0.4651	85.54%	0.2350	92.71%	12.47
5	0.3904	87.74%	0.2476	92.06%	12.47
6	0.3361	89.62%	0.1491	95.23%	12.42
7	0.3018	90.37%	0.1455	95.62%	12.48
8	0.2731	91.33%	0.1132	96.47%	24.20
9	0.2487	92.28%	0.1247	96.26%	12.59
10	0.2315	92.69%	0.1036	96.83%	12.70
11	0.2159	93.33%	0.1225	96.35%	12.64
12	0.2022	93.67%	0.0908	97.22%	12.56
13	**0.1943**	**94.04%**	**0.0700**	**98.03%**	**12.54**
14	0.1786	94.45%	0.1408	95.28%	12.48
15	0.1805	94.35%	0.0870	97.20%	12.43

The logged epoch times for epochs 2 to 15 add up to about 188 minutes. Epoch 8 took about 24 minutes, roughly double the others; the logs do not show why.

7.4 ResNet18 training history

The learning rate was halved to 0.0005 from epoch 7. The best checkpoint was saved at **epoch 10**.

Epoch	Train loss	Train acc.	Val loss	Val acc.	Time (min)
1	0.5380	87.02%	0.2311	93.19%	24.45
2	0.2102	93.69%	0.1774	94.24%	17.00
3	0.1646	94.93%	0.1478	95.34%	6.18
4	0.1462	95.46%	0.1375	95.60%	6.34
5	0.1350	95.59%	0.1274	95.62%	6.30
6	0.1242	95.97%	0.1334	95.60%	6.30
7	0.1147	96.21%	0.1424	95.50%	6.32
8	0.0960	96.81%	0.1066	96.42%	6.27
9	0.0923	96.99%	0.1147	96.26%	6.32
10	**0.0921**	**97.00%**	**0.1064**	**96.58%**	**6.57**

The logged epoch times add up to about 92 minutes. The first two epochs were much slower than the rest, and the notebooks do not establish a cause.

7.5 Learning curves

The four-panel figure `results/learning_curves.png` plots training loss, validation loss, training accuracy and validation accuracy for both models. ResNet18 starts higher and converges faster in training, while the custom CNN learns more slowly but reaches the lowest validation loss (0.0700) and highest validation accuracy (98.03%) at epoch 13. The custom CNN's validation curve is noisier, with a visible dip at epoch 14.

8. Results

8.1 Test-set evaluation

Both models were evaluated on the cleaned test set (10,704 images, 38 classes), loading the saved best checkpoint of each.

Metric	Custom CNN	ResNet18
Correct predictions	10,466	10,266
Incorrect predictions	238	438
Test accuracy	**97.73%**	95.91%

Metric	Custom CNN	ResNet18
Test loss	0.075071	0.129822
Macro precision	**97.21%**	95.46%
Macro recall	**96.65%**	94.54%
Macro F1-score	**96.85%**	94.90%
Best checkpoint epoch	13	10

8.2 Training, validation and test results at the best checkpoint

Metric	Custom CNN (epoch 13)	ResNet18 (epoch 10)
Training loss	0.1943	0.0921
Training accuracy	94.04%	97.00%
Validation loss	0.0700	0.1064
Validation accuracy	98.03%	96.58%
Test accuracy	97.73%	95.91%
Test loss	0.075071	0.129822

Test loss was calculated after training by evaluating each saved best checkpoint on the cleaned test set using CrossEntropyLoss. The reported test accuracies match the previously recorded results; the loss values are evaluation results from the same test loader.

8.3 Confusion matrices

A 38 × 38 confusion matrix was computed and plotted for each model with scikit-learn and Seaborn. The inspected figures are correctly titled “Custom CNN — Test Confusion Matrix” and “ResNet18 — Test Confusion Matrix,” respectively. The plotted matrices show strong diagonal concentration, consistent with the high overall test accuracies. The raw matrix values were not saved to the results files.

8.4 Per-class performance

Class-wise precision, recall, F1-score and support were produced with `classification_report` for both models. Selected classes are shown below.

Class	Support	Custom CNN P / R / F1	ResNet18 P / R / F1
Apple scab	127	1.00 / 0.95 / 0.98	0.99 / 0.87 / 0.92
Apple healthy	286	0.95 / 0.98 / 0.96	0.94 / 0.97 / 0.95

Class	Support	Custom CNN P / R / F1	ResNet18 P / R / F1
Corn Cercospora / Gray leaf spot	107	0.91 / 0.84 / 0.87	0.78 / 0.93 / 0.85
Potato healthy	32	1.00 / 0.75 / 0.86	0.97 / 0.88 / 0.92
Tomato Early blight	215	0.93 / 0.87 / 0.90	0.85 / 0.70 / 0.77
Tomato Late blight	372	0.98 / 0.88 / 0.93	0.87 / 0.95 / 0.91
Tomato healthy	328	0.99 / 0.99 / 0.99	0.97 / 0.96 / 0.97

These rows show why accuracy alone is not enough. Potato healthy has only 32 test images, and the custom CNN's recall on it (0.75) is well below its overall average.

8.5 Error analysis

The custom CNN made 238 errors, including confusion between visually similar tomato diseases and some healthy-leaf examples from different plants. ResNet18 made 438 errors, including Raspberry healthy images predicted as Cherry, Grape or Soybean healthy, and Potato late blight predicted as Tomato late blight or Tomato early blight. The notebooks record these errors but do not establish their precise causes.

9. Model Comparison

9.1 Performance

Metric	Custom CNN	ResNet18	Difference
Test accuracy	97.73%	95.91%	+1.82 points
Test loss	0.075071	0.129822	−0.054751 (lower for Custom CNN)
Macro precision	97.21%	95.46%	+1.75 points
Macro recall	96.65%	94.54%	+2.11 points
Macro F1-score	96.85%	94.90%	+1.95 points

The custom CNN was better on all four classification metrics and had a lower test loss (0.075071 versus 0.129822), with 200 more correct predictions on the same 10,704 test images.

9.2 Size

Property	Custom CNN	ResNet18
Total parameters	399,142	11,196,006

Property	Custom CNN	ResNet18
Parameters trained	399,142	19,494
Checkpoint size	about 4.60 MiB	about 42.94 MiB

The custom CNN is about 28 times smaller in total parameters.

9.3 Interpretation

Under the recorded split, preprocessing and training settings, the custom CNN outperformed the ResNet18 feature-extraction setup. This does not show that custom CNNs are better than ResNet18 in general. The models differed in architecture, augmentation, optimiser and scheduler, and ResNet18's backbone was frozen rather than fine-tuned. The comparison applies to these implementations only.

9.4 Overfitting and generalisation

Validation and test accuracy are close for both models (98.03% / 97.73% for the custom CNN, 96.58% / 95.91% for ResNet18), so no large gap appears. The custom CNN's training accuracy (94.04%) is below its validation accuracy because augmentation is applied only during training. The project did not test robustness to different cameras, backgrounds or field conditions.

10. Problems Faced and How They Were Handled

Problem	Action and outcome
Five exact duplicates across training and test	Found with MD5 hashes; the five test entries were removed, leaving 10,704 test images.
Strong class imbalance (152 to 5,507 images per class)	Macro metrics were reported. No class-weighted loss, weighted sampler or oversampling was used.
One RGBA image among RGB images	All images are converted to RGB on loading.
Compatibility of tensors, labels and model output	Batch shape, dtype, finite values, label range and output shape were checked; all checks passed.
Limited GPU memory (4 GB)	Batch size 16 was used, and a 200-batch pilot run preceded full custom CNN training. No out-of-memory error is recorded.
Kernel restarts and interrupted training	Checkpoints and histories were saved; the custom CNN resumed from epoch 2 and later restored its epoch-13 checkpoint for evaluation.
Metadata filename mismatch (`val_metadata.csv` vs `validation_metadata.csv`)	The correct file was located and all three splits loaded.

11. Software and Hardware

Component	Recorded value
Python	3.12.15 (Anaconda)
PyTorch / Torchvision (training notebooks)	2.13.0+cu130 / 0.28.0+cu130
Hugging Face datasets	5.1.0
NumPy / Pandas	2.5.4 / 3.1.0
Other libraries	scikit-learn, Matplotlib, Seaborn, Pillow, Hugging Face Hub (versions not recorded)
GPU	NVIDIA GeForce RTX 3050 4 GB Laptop GPU
Operating system	Windows

The preprocessing notebook ran in a separate CPU-only environment (PyTorch 2.14.1+cpu); the training notebooks used the CUDA environment above.

12. Limitations and Future Scope

12.1 Limitations

1. **Dataset domain.** PlantVillage images are leaf photographs; performance on real field images with complex backgrounds and lighting was not tested.
2. **Class imbalance.** Macro metrics were reported, but no imbalance-handling method was applied.
3. **Duplicates.** Only the five cross-split duplicates were confirmed removed; 21 duplicate groups were detected in the full dataset.
4. **Uncontrolled comparison.** The two models differ in several settings, and ResNet18 was not fine-tuned.
5. **External validation.** No independent dataset was used.
6. **Deployment.** No application or service was built; the output is a trained checkpoint and an evaluation workflow.
7. **Diagnostic certainty.** Predictions are dataset labels and are not a substitute for expert agricultural assessment.

12.2 Future scope

The following are proposals, not completed work.

- Fine-tune part or all of the ResNet18 backbone and compare it with the frozen version.
- Re-run the comparison with matched augmentation and training budgets.
- Try class-weighted loss, weighted sampling or other imbalance methods.
- Check all duplicate groups and detect near-duplicates.
- Evaluate on independent field-image datasets.
- Study the weakest classes (for example Potato healthy and Tomato Early blight).

- Build a prediction interface that shows the predicted class, confidence and limitations, and evaluate confidence calibration.

13. Conclusion

The project delivered a PlantVillage classification workflow covering dataset analysis, duplicate removal, preprocessing, training of two models, checkpointing, test evaluation, confusion matrices, class-wise reports, error inspection and model comparison. On the cleaned test set, the custom CNN achieved 97.73% accuracy, 0.075071 test loss and 96.85% macro F1-score, ahead of ResNet18 at 95.91% accuracy, 0.129822 test loss and 94.90% macro F1-score, while using far fewer parameters. These results hold for the specific configurations used, in which ResNet18's backbone was frozen and the two models used different augmentation and optimisation settings. External field-image validation and deployment remain future work.

References

[1] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419. https://doi.org/10.3389/fpls.2016.01419

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. https://doi.org/10.1109/CVPR.2016.90

[3] PyTorch Contributors. (n.d.). torch.nn.CrossEntropyLoss documentation. https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[4] PyTorch Contributors. (n.d.). Torchvision ResNet18 model documentation. https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

[5] scikit-learn Developers. (n.d.). Classification metrics documentation. https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

Appendix A. Project Files

File	Purpose
models/custom_cnn/best_model.pth	Best custom CNN checkpoint (epoch 13)
models/custom_cnn/training_history.csv	Custom CNN epoch-level history
models/resnet18/best_resnet18.pth	Best ResNet18 checkpoint (epoch 10)
models/resnet18/training_history.csv	ResNet18 epoch-level history
results/model_comparison.csv	Test-metric comparison table
results/learning_curves.png	Combined learning curves
results/final_results_summary.txt	Text summary of the comparison
notebooks/	Dataset analysis, EDA, preprocessing, custom CNN training and ResNet18 transfer-learning notebooks
results/ custom_cnn_confusion_matrix_notebook_screenshot.png	Custom CNN confusion-matrix notebook screenshot
results/ resnet18_confusion_matrix_notebook_screenshot.png	ResNet18 confusion-matrix notebook screenshot

Appendix B. Core Implementation Snippets

B.1 ResNet18 transfer-learning setup

```
weights = models.ResNet18_Weights.DEFAULT
model = models.resnet18(weights=weights)
model.fc = nn.Linear(model.fc.in_features, 38)
for param in model.parameters():
    param.requires_grad = False
for param in model.fc.parameters():
    param.requires_grad = True
```

The pretrained backbone is kept and frozen, and only the new 38-class output layer is trained.

B.2 Training step

```
optimizer.zero_grad(set_to_none=True)
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

Each step clears old gradients, computes class scores, calculates cross-entropy loss, back-propagates and updates the trainable parameters.

B.3 Evaluation metrics

```
accuracy  = accuracy_score(all_labels, all_predictions)
precision = precision_score(all_labels, all_predictions, average="macro", zero_division=0)
recall    = recall_score(all_labels, all_predictions, average="macro", zero_division=0)
f1        = f1_score(all_labels, all_predictions, average="macro", zero_division=0)
```

Macro averaging computes each metric per class and then averages, so every class counts equally despite the imbalance.